

```

const express = require('express');
const { Pool } = require('pg');
const cors = require('cors');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());

// Database connection
const pool = new Pool({
  connectionString: process.env.DATABASE_URL || 'postgresql://neondb_owner:npg_pGAUB9oIq4SF@ep-calm-sound-a8j0n5yd-pooler.eastus2.azure.neon.tech/neondb?sslmode=require',
  ssl: { rejectUnauthorized: false }
});

// Initialize database
const initDB = async () => {
  try {
    await pool.query(`
      CREATE TABLE IF NOT EXISTS families (
        id SERIAL PRIMARY KEY,
        family_code VARCHAR(50) UNIQUE NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
      CREATE TABLE IF NOT EXISTS family_members (
        id SERIAL PRIMARY KEY,
        family_code VARCHAR(50) REFERENCES families(family_code) ON DELETE CASCADE,
        name VARCHAR(255) NOT NULL,
        role VARCHAR(50) NOT NULL,
        joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
      CREATE TABLE IF NOT EXISTS tree_members (
        id SERIAL PRIMARY KEY,
        family_code VARCHAR(50) REFERENCES families(family_code) ON DELETE CASCADE,
        name VARCHAR(255) NOT NULL,
        relationship VARCHAR(255),
        birth_date DATE,
        bio TEXT,
        created_by VARCHAR(255),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
      CREATE TABLE IF NOT EXISTS stories (
        id SERIAL PRIMARY KEY,
        family_code VARCHAR(50) REFERENCES families(family_code) ON DELETE CASCADE,
        title VARCHAR(255) NOT NULL,
        author VARCHAR(255),
    `
  )
}

```

```

        content TEXT NOT NULL,
        tags TEXT,
        created_by VARCHAR(255),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );
CREATE TABLE IF NOT EXISTS events (
    id SERIAL PRIMARY KEY,
    family_code VARCHAR(50) REFERENCES families(family_code) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    date DATE NOT NULL,
    location VARCHAR(255),
    description TEXT,
    created_by VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
`);
console.log('✅ Database initialized successfully');
} catch (error) {
    console.error('❌ Database initialization error:', error);
}
};

initDB();

// Health check endpoint
app.get('/api/health', (req, res) => {
    res.json({ status: 'ok', message: 'Family Legacy API is running' });
});

// Create family
app.post('/api/families', async (req, res) => {
    const { familyCode, userName } = req.body;
    try {
        await pool.query('BEGIN');
        await pool.query('INSERT INTO families (family_code) VALUES ($1)', [familyCode]);
        await pool.query('INSERT INTO family_members (family_code, name, role) VALUES ($1, $2, $3)',
[familyCode, userName, 'admin']);
        await pool.query('COMMIT');
        res.json({ success: true, familyCode });
    } catch (error) {
        await pool.query('ROLLBACK');
        console.error('Error creating family:', error);
        res.status(500).json({ error: 'Failed to create family' });
    }
});

// Join family
app.post('/api/families/join', async (req, res) => {
    const { familyCode, userName } = req.body;
    try {

```

```

const check = await pool.query('SELECT * FROM families WHERE family_code = $1',
[familyCode]);
    if (check.rows.length === 0) {
        return res.status(404).json({ error: 'Family not found' });
    }
    await pool.query('INSERT INTO family_members (family_code, name, role) VALUES ($1, $2, $3)',
[familyCode, userName, 'member']);
    res.json({ success: true });
} catch (error) {
    console.error('Error joining family:', error);
    res.status(500).json({ error: 'Failed to join family' });
}
});

// Get family members
app.get('/api/families/:familyCode/members', async (req, res) => {
    try {
        const result = await pool.query('SELECT * FROM family_members WHERE family_code = $1 ORDER
BY joined_at DESC', [req.params.familyCode]);
        res.json(result.rows);
    } catch (error) {
        console.error('Error fetching members:', error);
        res.status(500).json({ error: 'Failed to fetch members' });
    }
});

// Get tree members
app.get('/api/families/:familyCode/tree', async (req, res) => {
    try {
        const result = await pool.query('SELECT * FROM tree_members WHERE family_code = $1 ORDER BY
created_at DESC', [req.params.familyCode]);
        res.json(result.rows);
    } catch (error) {
        console.error('Error fetching tree:', error);
        res.status(500).json({ error: 'Failed to fetch tree' });
    }
});

// Add tree member
app.post('/api/families/:familyCode/tree', async (req, res) => {
    const { name, relationship, birthDate, bio, createdBy } = req.body;
    try {
        const result = await pool.query(
            'INSERT INTO tree_members (family_code, name, relationship, birth_date, bio, created_by)
VALUES ($1, $2, $3, $4, $5, $6) RETURNING *',
            [req.params.familyCode, name, relationship, birthDate, bio, createdBy]
        );
        res.json(result.rows[0]);
    } catch (error) {
        console.error('Error adding tree member:', error);
    }
});

```

```

    res.status(500).json({ error: 'Failed to add tree member' });
  }
});

// Update tree member
app.put('/api/families/:familyCode/tree/:id', async (req, res) => {
  const { name, relationship, birthDate, bio } = req.body;
  try {
    const result = await pool.query(
      'UPDATE tree_members SET name = $1, relationship = $2, birth_date = $3, bio = $4 WHERE id
= $5 RETURNING *',
      [name, relationship, birthDate, bio, req.params.id]
    );
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Tree member not found' });
    }
    res.json(result.rows[0]);
  } catch (error) {
    console.error('Error updating tree member:', error);
    res.status(500).json({ error: 'Failed to update' });
  }
});

// Delete tree member
app.delete('/api/families/:familyCode/tree/:id', async (req, res) => {
  try {
    const result = await pool.query('DELETE FROM tree_members WHERE id = $1 RETURNING *',
[req.params.id]);
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Tree member not found' });
    }
    res.json({ success: true });
  } catch (error) {
    console.error('Error deleting tree member:', error);
    res.status(500).json({ error: 'Failed to delete' });
  }
});

// Get stories
app.get('/api/families/:familyCode/stories', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM stories WHERE family_code = $1 ORDER BY
created_at DESC', [req.params.familyCode]);
    res.json(result.rows);
  } catch (error) {
    console.error('Error fetching stories:', error);
    res.status(500).json({ error: 'Failed to fetch stories' });
  }
});

```

```

// Add story
app.post('/api/families/:familyCode/stories', async (req, res) => {
  const { title, author, content, tags, createdBy } = req.body;
  try {
    const result = await pool.query(
      'INSERT INTO stories (family_code, title, author, content, tags, created_by) VALUES ($1, $2, $3, $4, $5, $6) RETURNING *',
      [req.params.familyCode, title, author, content, tags, createdBy]
    );
    res.json(result.rows[0]);
  } catch (error) {
    console.error('Error adding story:', error);
    res.status(500).json({ error: 'Failed to add story' });
  }
});

// Update story
app.put('/api/families/:familyCode/stories/:id', async (req, res) => {
  const { title, author, content, tags } = req.body;
  try {
    const result = await pool.query(
      'UPDATE stories SET title = $1, author = $2, content = $3, tags = $4 WHERE id = $5 RETURNING *',
      [title, author, content, tags, req.params.id]
    );
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Story not found' });
    }
    res.json(result.rows[0]);
  } catch (error) {
    console.error('Error updating story:', error);
    res.status(500).json({ error: 'Failed to update story' });
  }
});

// Delete story
app.delete('/api/families/:familyCode/stories/:id', async (req, res) => {
  try {
    const result = await pool.query('DELETE FROM stories WHERE id = $1 RETURNING *',
[req.params.id]);
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Story not found' });
    }
    res.json({ success: true });
  } catch (error) {
    console.error('Error deleting story:', error);
    res.status(500).json({ error: 'Failed to delete' });
  }
});

```

```

// Get events
app.get('/api/families/:familyCode/events', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM events WHERE family_code = $1 ORDER BY date
ASC', [req.params.familyCode]);
    res.json(result.rows);
  } catch (error) {
    console.error('Error fetching events:', error);
    res.status(500).json({ error: 'Failed to fetch events' });
  }
});

// Add event
app.post('/api/families/:familyCode/events', async (req, res) => {
  const { name, date, location, description, createdBy } = req.body;
  try {
    const result = await pool.query(
      'INSERT INTO events (family_code, name, date, location, description, created_by) VALUES
($1, $2, $3, $4, $5, $6) RETURNING *',
      [req.params.familyCode, name, date, location, description, createdBy]
    );
    res.json(result.rows[0]);
  } catch (error) {
    console.error('Error adding event:', error);
    res.status(500).json({ error: 'Failed to add event' });
  }
});

// Update event
app.put('/api/families/:familyCode/events/:id', async (req, res) => {
  const { name, date, location, description } = req.body;
  try {
    const result = await pool.query(
      'UPDATE events SET name = $1, date = $2, location = $3, description = $4 WHERE id = $5
RETURNING *',
      [name, date, location, description, req.params.id]
    );
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Event not found' });
    }
    res.json(result.rows[0]);
  } catch (error) {
    console.error('Error updating event:', error);
    res.status(500).json({ error: 'Failed to update event' });
  }
});

// Delete event
app.delete('/api/families/:familyCode/events/:id', async (req, res) => {
  try {

```

```
    const result = await pool.query('DELETE FROM events WHERE id = $1 RETURNING *',
[req.params.id]);
    if (result.rows.length === 0) {
        return res.status(404).json({ error: 'Event not found' });
    }
    res.json({ success: true });
} catch (error) {
    console.error('Error deleting event:', error);
    res.status(500).json({ error: 'Failed to delete' });
}
});

// Start server
app.listen(PORT, () => {
    console.log(`🚀 Family Legacy API running on port ${PORT}`);
    console.log(`📍 Health check: http://localhost:${PORT}/api/health`);
});
```